NGINX Web Server

Ambient intelligence course (A.Y. 2025/2026)
Topic taught by Prof. Floriano Scioscia

Gabriele Colapinto

May 3, 2026

License

Contents

1 Nginx: An Event-Driven Alternative to the Apache Web Server

Nginx (pronounced "engine-x") is an open-source web server that was developed specifically to address the performance limitations of traditional servers like Apache, particularly in handling a high number of concurrent connections. It has gained immense popularity for its high performance, resource efficiency, and powerful capabilities as a reverse proxy and load balancer. Its architecture also enables key operational advantages, such as the ability to perform configuration reloads and "graceful stops" without dropping active connections.

1.1 A Fundamentally Different Architecture

The primary difference between Nginx and Apache lies in their core architectural models. Nginx employs an **asynchronous, event-based, non-blocking architecture**. It operates with a master process that is responsible for reading and validating the configuration and managing the lifecycle of a small, fixed number of worker processes. These workers, often configured to match the number of CPU cores, handle the actual connections and can each manage thousands of concurrent requests simultaneously.

This is in stark contrast to Apache's older, synchronous model, which traditionally assigned a new process or thread to each connection, leading to high memory consumption and performance degradation under heavy load. The direct impact of Nginx's architecture is significant: it uses substantially less memory and delivers superior performance at high levels of concurrency, making it an excellent choice for modern, high-traffic web applications.

1.2 The Nginx Configuration Model

Like Apache, Nginx uses a text-based configuration file. Its structure is composed of simple directives (a name and parameters ending with a semicolon) and block directives (contexts enclosed in curly braces). The configuration is organized into a clear hierarchy of three primary contexts:

1. `http`: The top-level context for all web traffic configuration. It contains one or more `server` blocks.
2. `server`: This block defines a specific virtual host, specifying the port to listen on and the domain name it should respond to.
3. `location`: Contained within a `server` block, a `location` block applies directives to requests for a specific URI path. Nginx processes a request by selecting the most specific matching location block.

This hierarchical structure is illustrated in the example below:

```
http {
    server {
        listen 80;
        server_name www.example.com;

        location / {
            root /data/www;
        }

        location /images/ {
            root /data;
        }
    }
}
```

This example shows a top-level `http` block containing a `server` block for a virtual host. Inside, two `location` blocks define different document roots for different URI paths, with `/images/` being more specific than the root path `/`. Directives in more specific contexts override those in more general ones.

1.3 Common Nginx Use Cases

The following examples demonstrate two of the most fundamental Nginx configurations.

1. **Virtual Host Configuration**: This `server` block configures a complete virtual host.
2. Here, `listen 4000;` instructs Nginx to accept connections on port 4000. `server_name www.example.com;` ensures this block only responds to requests with that specific `Host` header. The `root` directive sets the document root directory, and `index` specifies the default file to serve if a directory is requested.
3. **Reverse Proxy Configuration**: This `location` block uses the `proxy_pass` directive to forward all incoming requests to a backend server.
4. This configuration forms the basis of using Nginx as a reverse proxy or load balancer. The architectural pattern is identical to that discussed in the Apache section (2.4.4)—a gateway forwarding requests to backend servers—with only the syntax and underlying process model differing.

While their syntax and architecture differ, both Apache and Nginx provide powerful tools for implementing key web architecture patterns. One of the most critical of these is load balancing, a technique essential for building scalable and reliable services.

2 A Deeper Dive into Load Balancing

Load balancing is the process of distributing incoming tasks or network traffic across a set of resources, such as a cluster of servers. While reverse proxying is the *mechanism*, load balancing is the strategic *application* of that mechanism to achieve high availability and scalability—two pillars of modern web architecture. The primary goal is to optimize response time, maximize throughput, and prevent any single resource from becoming overloaded. Both Apache and Nginx can function as powerful software load balancers, making this a foundational strategy for building resilient web services.

2.1 Core Concepts and Scheduling Algorithms

Load balancing algorithms determine how tasks are distributed among the available servers. They can be broadly categorized into two types.

1. **Static vs. Dynamic Algorithms**: Static algorithms distribute tasks based on a fixed set of rules, without considering the current state or load of the servers. Dynamic algorithms, in contrast, are more sophisticated and adapt to the real-time load of each server, potentially moving tasks from an overloaded node to an underloaded one.
2. **Common Scheduling Algorithms**: Several common static algorithms are used for their simplicity and effectiveness:
    - **Round-Robin**: Requests are distributed to servers in a sequential, rotating order. The first request goes to the first server, the second to the second, and so on, cycling back to the beginning after the last server is reached.
    - **Randomized Static**: Tasks are assigned to servers at random. With a large number of requests, this method achieves a reasonably even distribution across the server pool.

2.2 Implementation Strategies

For internet services, load balancing can be implemented using two main strategies.

1. **DNS-Based Load Balancing**: This technique, often called Round-Robin DNS, associates a single domain name with multiple IP addresses. When a client requests the domain, the DNS server returns the IP addresses in a rotating order. While simple, this method can be unreliable due to DNS caching.
2. **Server-Side Load Balancers**: This is the more common and robust approach. A dedicated hardware appliance or a software program (such as Apache or Nginx in reverse proxy mode) acts as a single entry point for all client traffic. This load balancer receives requests and intelligently forwards them to one of several backend servers, making the backend infrastructure transparent to the client.

2.3 The Challenge of Persistence

In a load-balanced environment, a significant challenge arises with managing user sessions.

1. **The Problem**: If a user's session data (e.g., items in a shopping cart) is stored locally on one backend server, a problem occurs if the load balancer sends their next request to a different server. That new server will have no knowledge of the user's session, leading to a broken user experience.
2. **The Solution ("Stickiness")**: The solution is **persistence**, often called "stickiness." This is a technique where the load balancer ensures that all requests from a single user's session are consistently routed to the same backend server. This can be achieved by tracking the client's IP address or other identifiers, thereby maintaining session state.

Managing state is a key consideration when designing any load-balanced architecture, as it directly impacts both user experience and system reliability.

3 Fundamentals of NGINX Configuration Architecture

The strategic importance of NGINX lies in its modular configuration architecture, which provides granular control over web traffic and resource management. This design serves as the robust foundation for modern web infrastructure, allowing architects to define precise rules for how an instance handles various types of requests and system resources. This modularity ensures that the server can scale from a simple file provider to a complex traffic gateway without a total redesign of the underlying configuration logic.

NGINX configuration is built upon two distinct types of directives:

- **Simple Directives:** Consist of a single line of text (e.g., `root /data/www;`) and must be terminated with a semicolon.
- **Block Directives:** Defined by a name followed by a set of directives enclosed within curly braces `{ }`. These create a "context" for the directives within them.

The architecture is organized into a visual and functional hierarchy:

1. **HTTP:** The outermost context for global settings applicable to all web traffic.
2. **Server:** Managed within the HTTP context to define settings for individual Virtual Hosts.
3. **Location:** Nested within the Server context to define how NGINX handles specific resource paths, often utilizing regular expressions or prefixes.

This structure follows a strict hierarchical application logic (**HTTP > Server > Location**). In this model, the most specific context—the Location block—is capable of overriding more general settings defined in the Server or HTTP contexts. This allows for highly specialized resource handling without compromising global stability. This foundational architecture provides the flexibility required for implementing advanced features like Virtual Hosting.

4 Virtual Hosting and Resource Mapping

Virtual Hosting is a strategic capability that enables a single physical NGINX instance to host multiple distinct domains and services efficiently. This optimization allows organizations to maximize hardware utilization by running diverse web applications through a unified gateway.

To establish a Virtual Host, four essential directives are utilized within the `server` block:

- `listen`: Defines the TCP port for incoming traffic (e.g., `listen 80;`).
- `server_name`: Identifies the host name or domain assigned to the virtual host (e.g., `server_name example.com;`).
- `root`: Maps the request to a specific path on the local file system.
- `index`: Defines the default files (like `index.html`) to serve when a directory is requested.

4.1 Scenario: Path Resolution and Matching Logic

NGINX resolves overlapping paths based on prefix specificity and order. If a request matches multiple contexts at the same hierarchical level, the last defined context is applied.

```
server {
    listen 80;
    server_name localhost;

    # Context A: Generic root
    location / {
        root /data/www;
    }

    # Context B: Specific prefix
    location /images/ {
        root /data;
    }
}
```

Architectural Analysis: If a client requests `/images/example.png`, the request matches both `/` and `/images/`. Because they share the same hierarchical level, NGINX applies the last defined context (Context B). Consequently, the file is sought at `/data/images/example.png`. It is critical to order configurations by placing generic locations first and specific locations afterward to ensure correct resource delivery.

This ability to map local resources effectively serves as the precursor to NGINX's more advanced role in intermediate traffic handling.

5 The Reverse Proxy Implementation

Historically, NGINX's shift from a simple web server to a high-performance reverse proxy was driven by its superior efficiency over legacy systems like Apache, particularly regarding connection concurrency. As a proxy, NGINX acts as an intermediary, receiving client requests and forwarding them to backend services.

The primary directive for this role is `proxy_pass`. It directs traffic to an internal server or a containerized instance. In modern microservices, architects often use special DNS names to bridge network layers:

```
location / {
    proxy_pass http://host.docker.internal:8080;
}
```

Note: `host.docker.internal` *is a special DNS name that resolves to the internal IP address used by the host, facilitating seamless communication between containers and host-level services.*

In professional deployments, NGINX often performs a hybrid "filtering" role. It serves static content directly (using `root`) while proxying all other requests to a backend. By offloading static asset delivery from the application server, NGINX significantly increases overall system throughput.

This single-target proxying capability provides the technical framework required for more complex multi-server distribution known as Load Balancing.

6 Advanced Load Balancing Strategies

Load balancing is implemented to maximize throughput, reduce latency, and ensure fault tolerance. Rather than "scaling up" with expensive hardware, NGINX facilitates a "scale-out" architecture, distributing traffic across a fleet of identical, standardized server instances.

Architects use the `upstream` context to group backend instances. Within this context, specific policies can be applied:

Policy Name	Directive	Mechanism	Use Case / Benefit
Round Robin	*(Default)*	Requests are distributed sequentially.	Standard distribution for i
Least Connected	`least_conn;`	Targets the server with the lowest active connections.	Prevents overloading busy
IP Hash	`ip_hash;`	Maps client IP to a specific server via a hash function.	Ensures session persistence

6.1 Implementation Example: Load Balancer with Persistence

```
http {
    upstream myapp1 {
        ip_hash; # Ensures session persistence
        server srv1.example.com;
        server srv2.example.com;
        server srv3.example.com;
    }

    server {
        listen 80;
        location / {
            proxy_pass http://myapp1;
        }
    }
}
```

By replicating server instances with identical configurations, NGINX provides high reliability. If one node fails, the load balancer continues to respond using the remaining healthy instances, ensuring seamless failover.

7 Containerization with Docker and Docker Compose

Containerizing NGINX ensures environment parity across development and production, simplifying deployment workflows. To build a custom image, a `Dockerfile` is used to package the configuration and static assets:

7.1 Custom NGINX Dockerfile

```
FROM nginx:latest
# Copy local configs and static content into the image
COPY ./nginx_conf/. /etc/nginx/conf.d/
COPY ./nginx_root/. /usr/share/nginx/html/
# Define accessible container ports
EXPOSE 80 4000 5000
```

To manage the deployment and persistence of these containers, `docker-compose.yml` defines the service parameters, specifically port mapping and volume externalization:

7.2 Docker Compose Service (docker-compose.yml)

```
version: '3'
services:
  web:
    image: nginx-test
    build: .
    ports:
      - "8080:80"   # Maps Host Port 8080 to Container Port 80
      - "5000:5000"
    volumes:
      - nginx_root:/usr/share/nginx/html
      - nginx_conf:/etc/nginx/conf.d
volumes:
  nginx_root:
    external: true
  nginx_conf:
    external: true
```

The `ports` mapping (e.g., `8080:80`) is crucial: it allows external traffic on the host's port 8080 to be routed into the container's standard web port. This ensures the service is accessible while maintaining isolation.

8 Comparative Performance Analysis: NGINX vs. Apache HTTP Server

The competition between NGINX and Apache 2.4 has defined modern web performance standards. While Apache has introduced a new architecture to improve its efficiency, key differences remain based on the application's resource profile:

- **Concurrency:** NGINX is engineered to handle extreme levels of simultaneous connections (concurrency) far more efficiently than Apache.
- **Static vs. Dynamic:** NGINX is significantly faster at serving static resources (images, video, music). Conversely, Apache has historical strengths in handling dynamic content and complex business logic.

8.1 The Ideal Hybrid Configuration

The industry-standard architectural solution is to combine both systems to leverage their respective strengths. In this hybrid model, NGINX serves as the front-facing server:

1. **NGINX** handles all requests for static assets directly from the file system.
2. **NGINX** acts as a reverse proxy for dynamic requests, forwarding them to an **Apache** instance running on the internal network to process business logic.

Choosing between these systems—or implementing them in tandem—should be based on the nature of the application: static-heavy workloads favor NGINX, while applications requiring extensive dynamic data generation via AJAX or complex logic may still utilize Apache as the backend engine.